

第九模块：系统架构与长期运行工程设计（专家级超详细版 · 装备方向）

本模块不是讲设计模式，也不是讲理论架构图。

目标是：

★ 让系统可以稳定演进多年，而不是越改越乱。

本模块属于整个体系的顶层能力，总结前面所有底层模块，形成真正工程架构思维。

一、为什么很多嵌入式项目越维护越崩

真实原因通常不是代码能力，而是架构问题：

- 业务逻辑与驱动耦合
- ISR 里写业务代码
- 全局变量泛滥
- 通信层直接控制硬件
- 升级后无法兼容旧版本

长期结果：

👉 新需求只能不断打补丁。

二、稳定架构的核心原则（装备级）

2.1 分层不是为了好看，而是为了隔离变化

推荐层次：

- BSP（底层驱动）
- Service（功能服务层）
- Logic（业务逻辑层）
- Interface（通信/人机接口）

关键原则：

上层不能直接访问寄存器。

2.2 数据流优先，而不是函数调用优先

很多系统设计成：

函数互相调用 → 形成耦合网。

推荐方式：

数据流驱动（Event / Message / Queue）。

三、任务划分的真正标准（不是随便分线程）

任务划分建议基于：

- 时序要求
- 数据频率
- 阻塞行为

典型结构：

- Control Task（高实时）
- Comm Task（中优先级）
- Log Task（低优先级）

不要按“模块名称”划分任务。

四、接口设计（防止系统越改越乱）

4.1 不要暴露底层结构体

错误示例：

把驱动结构体直接暴露给业务层。

正确方式：

通过 API 或消息结构传递。

4.2 版本兼容设计

通信协议必须预留：

- Version 字段

- Feature Bit

否则升级后设备无法互通。

五、配置与参数管理（长期运行关键）

推荐设计：

- Flash 参数区
- 默认参数 + 用户参数
- 参数CRC

升级时：

不要覆盖用户参数。

六、日志与远程维护架构（真正工业级要求）

系统必须支持：

- 错误码统计
- 运行时间统计
- 重启原因记录

原因：

现场无法连接调试器。

七、模块解耦的真实方法（很多人理解错）

解耦不是多写接口文件，而是：

👉 限制依赖方向。

原则：

- 业务层不能依赖驱动实现细节
 - 驱动层不知道业务存在
-

八、内存与资源规划（专家级思维）

系统设计时必须预留：

- RAM 冗余（至少 20 %）
- Flash 冗余（用于升级）
- 任务栈余量

不要把资源用满再上线。

九、长期稳定性的真正指标（不是是否崩溃）

一个成熟系统应该关注：

- 平均CPU占用率
- 最大延迟（Worst-case latency）
- 错误恢复次数
- DMA溢出率

这些指标必须可观测。

十、高频架构考点（标准答案）

Q 1：为什么业务层不能直接操作寄存器？

标准答案：

会造成耦合，后期更换硬件或修改驱动成本极高。

Q 2：为什么推荐事件驱动而不是函数互调？

标准答案：

事件驱动可以降低模块耦合，提高扩展性。

Q 3：为什么升级必须预留Flash空间？

标准答案：

用于双备份与回滚，否则升级风险极高。

Q 4：长期运行系统最重要的能力是什么？

标准答案：

可观测性与可恢复性，而不是一次性性能。

十一、工程注意事项（专家经验）

- 1）架构设计先于功能开发。
 - 2）不要在中断中写业务逻辑。
 - 3）所有模块必须可替换。
 - 4）协议必须可扩展。
 - 5）系统必须支持远程诊断。
-

十二、深度练习（强烈建议）

- 画出你的系统分层图并检查依赖方向。
 - 统计当前CPU与RAM使用率并评估余量。
 - 重构一个模块为事件驱动结构。
-

完成本模块后，你将具备“系统级架构设计能力”，这是高级嵌入式工程师真正的核心竞争力。